

Lab 8: Play Tennis Classification

This notebook implements **Categorical Naive Bayes** on the Play Tennis dataset, evaluates it using a held-out test set, predicts the outcome for a new weather condition, and compares its performance with **Decision Tree** and **Logistic Regression**.

1. Import the required libraries

```
In [1]: from pathlib import Path

import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    precision_recall_fscore_support,
)
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import CategoricalNB
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, OrdinalEncoder
from sklearn.tree import DecisionTreeClassifier

RANDOM_STATE = 42
```

2. Load and inspect the dataset

```
In [2]: DATASET_PATH = Path('/mnt/data/college/4_tri/ML-Lab-2547121/datasets/Lab 8 -
data = pd.read_csv(DATASET_PATH)

print(f"Dataset shape: {data.shape}")
print("\nMissing values per column:")
print(data.isna().sum())
data
```

Dataset shape: (50, 6)

Missing values per column:

No	0
Outlook	0
Temperature	0
Humidity	0
Wind	0
Play Tennis	0

dtype: int64

Out[2]:

	No	Outlook	Temperature	Humidity	Wind	Play Tennis
0	1	Sunny	Hot	High	Weak	No
1	2	Sunny	Hot	High	Strong	No
2	3	Overcast	Hot	High	Weak	Yes
3	4	Rain	Mild	High	Weak	Yes
4	5	Rain	Cool	Normal	Weak	Yes
5	6	Rain	Cool	Normal	Strong	No
6	7	Overcast	Cool	Normal	Strong	Yes
7	8	Sunny	Mild	High	Weak	No
8	9	Sunny	Cool	Normal	Weak	Yes
9	10	Rain	Mild	Normal	Weak	Yes
10	11	Sunny	Mild	Normal	Strong	Yes
11	12	Overcast	Mild	High	Strong	Yes
12	13	Overcast	Hot	Normal	Weak	Yes
13	14	Rain	Mild	High	Strong	No
14	15	Sunny	Cool	High	Weak	No
15	16	Sunny	Cool	High	Strong	No
16	17	Sunny	Hot	Normal	Weak	Yes
17	18	Sunny	Hot	Normal	Strong	Yes
18	19	Overcast	Cool	High	Weak	Yes
19	20	Overcast	Cool	High	Strong	Yes
20	21	Overcast	Mild	Normal	Weak	Yes
21	22	Overcast	Mild	Normal	Strong	Yes
22	23	Rain	Hot	High	Weak	Yes
23	24	Rain	Hot	High	Strong	No
24	25	Rain	Hot	Normal	Weak	Yes
25	26	Rain	Hot	Normal	Strong	No
26	27	Rain	Cool	High	Weak	Yes
27	28	Rain	Cool	High	Strong	No
28	29	Rain	Mild	Normal	Strong	Yes

	No	Outlook	Temperature	Humidity	Wind	Play Tennis
29	30	Sunny	Mild	High	Strong	No
30	31	Sunny	Cool	Normal	Strong	Yes
31	32	Sunny	Hot	High	Weak	No
32	33	Sunny	Hot	Normal	Weak	Yes
33	34	Sunny	Mild	Normal	Weak	Yes
34	35	Overcast	Hot	High	Strong	Yes
35	36	Overcast	Hot	Normal	Strong	Yes
36	37	Overcast	Cool	Normal	Weak	Yes
37	38	Overcast	Mild	High	Weak	Yes
38	39	Rain	Mild	High	Weak	Yes
39	40	Rain	Cool	Normal	Weak	Yes
40	41	Rain	Hot	High	Strong	No
41	42	Sunny	Cool	High	Weak	No
42	43	Sunny	Mild	Normal	Strong	Yes
43	44	Sunny	Hot	High	Strong	No
44	45	Overcast	Cool	High	Weak	Yes
45	46	Overcast	Mild	Normal	Strong	Yes
46	47	Rain	Mild	Normal	Weak	Yes
47	48	Rain	Cool	High	Strong	No
48	49	Sunny	Cool	Normal	Weak	Yes
49	50	Overcast	Hot	Normal	Weak	Yes

3. Separate the input features and target

`No` is only a row identifier, so it is excluded from the input features. The target labels are converted to integers with `LabelEncoder`.

```
In [3]: X = data.drop(columns=["No", "Play Tennis"])
        y = data["Play Tennis"]

        target_encoder = LabelEncoder()
        y_encoded = target_encoder.fit_transform(y)

        target_mapping = {
```

```

class_name: int(code)
for code, class_name in enumerate(target_encoder.classes_)
}

print("Input features:", list(X.columns))
print("Target mapping:", target_mapping)
print("\nClass distribution:")
print(y.value_counts())

```

Input features: ['Outlook', 'Temperature', 'Humidity', 'Wind']
 Target mapping: {'No': 0, 'Yes': 1}

Class distribution:
 Play Tennis
 Yes 34
 No 16
 Name: count, dtype: int64

4. Divide the data into training and testing sets

An 80:20 split is used. `stratify` preserves the class proportions, while the fixed random state makes the results reproducible.

```

In [4]: X_train, X_test, y_train, y_test = train_test_split(
        X,
        y_encoded,
        test_size=0.20,
        random_state=RANDOM_STATE,
        stratify=y_encoded,
    )

print(f"Training samples: {len(X_train)}")
print(f"Testing samples : {len(X_test)}")

```

Training samples: 40
 Testing samples : 10

5. Encode the categorical features

`CategoricalNB` expects each feature to contain non-negative integer category codes. The `OrdinalEncoder` is fitted **only on the training data** to avoid test-data leakage. Adding 1 reserves code 0 for any previously unseen category.

```

In [5]: nb_encoder = OrdinalEncoder(
        handle_unknown="use_encoded_value",
        unknown_value=-1,
    )

X_train_nb = nb_encoder.fit_transform(X_train).astype(int) + 1
X_test_nb = nb_encoder.transform(X_test).astype(int) + 1

encoded_preview = pd.DataFrame(

```

```

X_train_nb,
columns=X.columns,
index=X_train.index,
)
encoded_preview["Play Tennis"] = y_train

print("Categories learned from the training data:")
for feature, categories in zip(X.columns, nb_encoder.categories_):
    print(f"{feature}: {list(categories)}")

print("\nEncoded training data (first five rows):")
encoded_preview.head()

```

Categories learned from the training data:

Outlook: ['Overcast', 'Rain', 'Sunny']

Temperature: ['Cool', 'Hot', 'Mild']

Humidity: ['High', 'Normal']

Wind: ['Strong', 'Weak']

Encoded training data (first five rows):

Out[5]:

	Outlook	Temperature	Humidity	Wind	Play Tennis
14	3	1	1	2	0
0	3	2	1	2	0
2	1	2	1	2	1
21	1	3	2	1	1
29	3	3	1	1	0

6. Train and evaluate Categorical Naive Bayes

```

In [6]: categorical_nb = CategoricalNB()
categorical_nb.fit(X_train_nb, y_train)

y_pred_nb = categorical_nb.predict(X_test_nb)
nb_accuracy = accuracy_score(y_test, y_pred_nb)

class_codes = list(range(len(target_encoder.classes_)))
class_names = list(target_encoder.classes_)

print(f"Categorical Naive Bayes accuracy: {nb_accuracy:.2%}")

cm_nb = pd.DataFrame(
    confusion_matrix(y_test, y_pred_nb, labels=class_codes),
    index=[f"Actual {name}" for name in class_names],
    columns=[f"Predicted {name}" for name in class_names],
)
print("\nConfusion matrix:")
display(cm_nb)

print("Classification report:")

```

```

print(
    classification_report(
        y_test,
        y_pred_nb,
        labels=class_codes,
        target_names=class_names,
        zero_division=0,
    )
)

```

Categorical Naive Bayes accuracy: 90.00%

Confusion matrix:

	Predicted No	Predicted Yes
Actual No	2	1
Actual Yes	0	7

Classification report:

	precision	recall	f1-score	support
No	1.00	0.67	0.80	3
Yes	0.88	1.00	0.93	7
accuracy			0.90	10
macro avg	0.94	0.83	0.87	10
weighted avg	0.91	0.90	0.89	10

7. Predict the requested weather condition

```

In [7]: new_weather = pd.DataFrame(
    [
        {
            "Outlook": "Sunny",
            "Temperature": "Cool",
            "Humidity": "High",
            "Wind": "Strong",
        }
    ]
)

new_weather_nb = nb_encoder.transform(new_weather).astype(int) + 1
new_prediction_code = categorical_nb.predict(new_weather_nb)
new_prediction = target_encoder.inverse_transform(new_prediction_code)[0]
new_probabilities = categorical_nb.predict_proba(new_weather_nb)[0]

probability_table = pd.DataFrame(
    {
        "Class": target_encoder.inverse_transform(categorical_nb.classes_),
        "Probability": new_probabilities,
    }
)

probability_table["Probability (%)"] = (

```

```

probability_table["Probability"] * 100
).round(2)

print("Weather condition:")
display(new_weather)
print(f"Predicted class: {new_prediction}")
print("\nClass probabilities:")
display(probability_table)

```

Weather condition:

	Outlook	Temperature	Humidity	Wind
0	Sunny	Cool	High	Strong

Predicted class: No

Class probabilities:

	Class	Probability	Probability (%)
0	No	0.769347	76.93
1	Yes	0.230653	23.07

8. Compare with Decision Tree and Logistic Regression

For these two models, one-hot encoding is more appropriate because it avoids imposing an artificial order on weather categories. The same training and testing rows are retained for a fair comparison.

```

In [8]: onehot_encoder = OneHotEncoder(handle_unknown="ignore")
X_train_onehot = onehot_encoder.fit_transform(X_train)
X_test_onehot = onehot_encoder.transform(X_test)
new_weather_onehot = onehot_encoder.transform(new_weather)

decision_tree = DecisionTreeClassifier(random_state=RANDOM_STATE)
logistic_regression = LogisticRegression(
    max_iter=1000,
    random_state=RANDOM_STATE,
)

decision_tree.fit(X_train_onehot, y_train)
logistic_regression.fit(X_train_onehot, y_train)

model_inputs = {
    "Categorical Naive Bayes": (categorical_nb, X_test_nb, new_weather_nb),
    "Decision Tree": (decision_tree, X_test_onehot, new_weather_onehot),
    "Logistic Regression": (
        logistic_regression,
        X_test_onehot,
        new_weather_onehot,
    )
}

```

```

    ),
}

```

```

In [9]: comparison_rows = []
weather_prediction_rows = []

for model_name, (model, test_features, weather_features) in model_inputs.items():
    test_prediction = model.predict(test_features)
    precision, recall, f1, _ = precision_recall_fscore_support(
        y_test,
        test_prediction,
        average="weighted",
        zero_division=0,
    )
    comparison_rows.append(
        {
            "Model": model_name,
            "Accuracy": accuracy_score(y_test, test_prediction),
            "Weighted Precision": precision,
            "Weighted Recall": recall,
            "Weighted F1-score": f1,
        }
    )

    weather_code = model.predict(weather_features)
    weather_class = target_encoder.inverse_transform(weather_code)[0]
    weather_probabilities = model.predict_proba(weather_features)[0]
    weather_prediction_rows.append(
        {
            "Model": model_name,
            "Predicted Class": weather_class,
            **{
                f"P({class_name})": probability
                for class_name, probability in zip(
                    target_encoder.inverse_transform(model.classes_),
                    weather_probabilities,
                )
            },
        }
    )

comparison = (
    pd.DataFrame(comparison_rows)
    .set_index("Model")
    .sort_values("Accuracy", ascending=False)
)

weather_comparison = pd.DataFrame(weather_prediction_rows).set_index("Model")

print("Model performance on the test set:")
display(comparison.style.format("{:.2%}"))

print("Predictions for Sunny, Cool, High humidity, Strong wind:")
display(
    weather_comparison.style.format(
        {"P(No)": "{:.2%}", "P(Yes)": "{:.2%}"
    )
)

```



```

    )
)

best_accuracy = comparison["Accuracy"].max()
best_models = comparison.index[comparison["Accuracy"] == best_accuracy].tolist()
print(
    f"Highest test accuracy: {best_accuracy:.2%} "
    f"({','.join(best_models)})"
)

```

Model performance on the test set:

	Accuracy	Weighted Precision	Weighted Recall	Weighted F1-score
Model				
Decision Tree	100.00%	100.00%	100.00%	100.00%
Categorical Naive Bayes	90.00%	91.25%	90.00%	89.33%
Logistic Regression	90.00%	91.25%	90.00%	89.33%

Predictions for Sunny, Cool, High humidity, Strong wind:

	Predicted Class	P(No)	P(Yes)
Model			
Categorical Naive Bayes	No	76.93%	23.07%
Decision Tree	No	100.00%	0.00%
Logistic Regression	No	82.02%	17.98%

Highest test accuracy: 100.00% (Decision Tree)

Conclusion

The comparison uses only 10 test observations, so a single prediction changes accuracy by 10 percentage points. The reported results are useful for this lab dataset, but they should not be treated as a broad ranking of the algorithms. Cross-validation would provide a more stable comparison on such a small dataset.